

When you install Linux, you should create at least one regular user to be your personal account. For this chapter, you should log in as the regular user.

2.2.1 The Shell Window

After logging in, open a shell window (often referred to as a *terminal*). The easiest way to do so from a GUI like Gnome or KDE is to open a terminal application, which starts a shell inside a new window. Once you’ve opened a shell, it should display a prompt at the top that usually ends with a dollar sign (\$). On Ubuntu, that prompt should look like `name@host:path$`, and on Fedora, it’s `[name@host path]$`, where *name* is your username, *host* is the name of your machine, and *path* is your current working directory (see Section 2.4.1). If you’re familiar with Windows, the shell window will look something like a DOS command prompt; in macOS the Terminal application is essentially the same as a Linux shell window.

This book contains many commands that you will type at a shell prompt. They all begin with a single \$ to denote the shell prompt. For example, type this command (just the part in bold, not the \$) and press ENTER:

```
$ echo Hello there.
```

NOTE

Many shell commands in this book start with #. You should run these commands as the superuser (root), so they require extra caution. The best practice when running them is to use `sudo` in order to provide some protection and a log that you can look up later for possible errors. You’ll see how to do this in Section 2.20.

Now enter this command:

```
$ cat /etc/passwd
```

This command displays the contents of the `/etc/passwd` system information file and then returns your shell prompt. Don’t worry about what *this* file does right now; you’ll learn all about it in Chapter 7.

Commands usually begin with a program to run and may be followed by *arguments* that tell the program what to operate on and how to do so. Here, the program is `cat`, and there is one argument, `/etc/passwd`. Many arguments are options that modify the default behavior of a program and typically begin with a dash (-). You’ll see this shortly in the discussion of `ls`. There are some exceptions that don’t follow this normal command structure, however, such as shell built-ins and the temporary use of environment variables.

2.2.2 cat

The `cat` program is one of the easiest in Unix to understand; it simply outputs the contents of one or more files or another source of input. The general syntax of a `cat` command is as follows:

```
$ cat file1 file2 ...
```

When you run this command, `cat` prints the contents of *file1*, *file2*, and any other files that you specify as arguments (denoted by `...` in the preceding example), and then exits. The program is called `cat` because it performs concatenation when it prints the contents of more than one file. There are many ways to run `cat`; let’s use it to explore Unix I/O.

2.2.3 Standard Input and Standard Output

Unix processes use I/O *streams* to read and write data. Processes read data from input streams and write data to output streams. Streams are very flexible. For example, the source of an input stream can be a file, a device, a terminal window, or even the output stream from another process.

To see an input stream at work, enter `cat` (with no arguments) and press ENTER. This time, you won’t get any immediate output, and you won’t get your shell prompt back because `cat` is still running. Now type anything and press ENTER at the end of each line. When used like this, the `cat` command repeats any line that you type. Once you’re sufficiently bored, press CTRL-D on an empty line to terminate `cat` and return to the shell prompt.

The reason `cat` adopts an interactive behavior here has to do with streams. When you don’t specify an input filename, `cat` reads from the *standard input* stream provided by the Linux kernel rather than a stream connected to a file. In this case, the standard input is connected to the terminal where you run `cat`.

NOTE

Pressing CTRL-D on an empty line stops the current standard input entry from the terminal with an EOF (end-of-file) message (and often terminates a program). Don’t confuse this with CTRL-C, which usually terminates a program regardless of its input or output.

Standard output is similar. The kernel gives each process a standard output stream where it can write its output. The `cat` command always writes its output to the standard output. When you ran `cat` in the terminal, the standard output was connected to that terminal, so that’s where you saw the output.

Standard input and output are often abbreviated as *stdin* and *stdout*. Many commands operate as `cat` does; if you don't specify an input file, the command reads from `stdin`. Output is a little different. Some programs (like `cat`) send output only to `stdout`, but others have the option to send output directly to files.

There is a third standard I/O stream, called *standard error*. You'll see it in Section 2.14.1.

One of the best features of standard streams is that you can easily manipulate them to read and write to places other than the terminal, as you'll learn in Section 2.14. In particular, you'll learn how to connect streams to files and other processes.

2.3 Basic Commands

Now let's look at some more Unix commands. Most of the following programs take multiple arguments, and some have so many options and formats that an unabridged listing would be pointless. This is a simplified list of the basic commands; you don't need all of the details just yet.

2.3.1 *ls*

The `ls` command lists the contents of a directory. The default is the current directory, but you can add any directory or file as an argument, and there are many useful options. For example, use `ls -l` for a detailed (long) listing and `ls -F` to display file type information. Here is a sample long listing; it includes the owner of the file (column 3), the group (column 4), the file size (column 5), and the modification date/time (between column 5 and the filename):

```
$ ls -l
total 3616
-rw-r--r-- 1 juser users 3804    May 28 10:40  abusive.c
-rw-r--r-- 1 juser users 4165    Aug 13 10:01  battery.zip
-rw-r--r-- 1 juser users 131219  Aug 13 10:33  beav_1.40-13.tar.gz
-rw-r--r-- 1 juser users 6255    May 20 14:34  counTry.c
drwxr-xr-x 2 juser users 4096    Jul 17 20:00  cs335
-rwxr-xr-x 1 juser users 7108    Jun 16 13:05  dhry
-rw-r--r-- 1 juser users 11309   Aug 13 10:26  dhry.c
-rw-r--r-- 1 juser users 56      Jul  9 15:30  doit
drwxr-xr-x 6 juser users 4096    Feb 20 13:51  dw
drwxr-xr-x 3 juser users 4096    Jul  1 16:05  hough-stuff
```

You'll learn more about column 1 of this output in Section 2.17. You can ignore column 2 for now; it's the number of hard links to the file and is explained in Section 4.6.

2.3.2 *cp*

In its simplest form, `cp` copies files. For example, to copy *file1* to *file2*, enter this:

```
$ cp file1 file2
```

You can also copy a file to another directory, keeping the same file name in that directory:

```
$ cp file dir
```

To copy more than one file to a directory (folder) named *dir*, try something like this example, which copies three files:

```
$ cp file1 file2 file3 dir
```

2.3.3 *mv*

The `mv` (move) command works much like `cp`. In its simplest form, it renames a file. For example, to rename *file1* to *file2*, enter this:

```
$ mv file1 file2
```

You can also use `mv` to move files to other directories in the same way as `cp`.

2.3.4 *touch*

The `touch` command can create a file. If the target file already exists, `touch` doesn't change the file, but it does update the file's modification timestamp. For example, to create an empty file, enter this:

```
$ touch file
```

Then run `ls -l` on that file. You should see output like the following, where the date and time indicate when you ran `touch`:

```
$ ls -l file
-rw-r--r-- 1 juser users 0  May 21 18:32  file
```

To see a timestamp update, wait at least a minute and then run the same `touch` command again. The timestamp returned by `ls -l` will update.

2.3.5 *rm*

The `rm` command deletes (removes) a file. After you remove a file, it's usually gone from your system and generally cannot be undeleted unless you restore it from a backup.